



Sistemas Distribuidos

Grado en Ingeniería Informática

Ejercicio 2.

Grupo 81

Año académico 2025/2026

Juan Mayoral Fernández | 100524298 | 100524298@alumnos.uc3m.es
Jose Barrio Bielsa | 100522205 | 100522205@alumnos.uc3m.es

ÍNDICE

Fase 1: Diseño Protocolo Sockets TCP	2
Fase 2: Compilación	3
Fase 3: Tests	3

Fase 1: Diseño Protocolo Sockets TCP

Esta primera fase consiste en adaptar el ejercicio 1 (basado en colas de mensajes) a un protocolo basado en **Sockets TCP**. Al ser necesario una dirección IP y un puerto para ejecutar el programa hemos creado variables de entorno (**IP_TUPLAS=localhost** y **PORT_TUPLAS=8080**) siguiendo las especificaciones del enunciado.

En este ejercicio hemos realizado cambios únicamente a las partes del proyecto que no necesitábamos para el protocolo. Es decir, todos los archivos que gestionaban la cola de mensajes han sido sustituidos por su análogo para **Sockets TCP**:

Ejercicio 1	Ejercicio 2
mq.h	sock.h sock.c
servidor-mq.c	servidor-sock.c
proxy-mq.c	proxy-sock.c

En **sock.h** están declaradas las estructuras utilizadas en el ejercicio 1 **Peticion** y **Respuesta**. Además, se han declarado también las funciones de envío y recepción para sockets (**sock_send** y **sock_receive**), así como funciones complementarias para pasar los distintos tipos de C a formato string (**int_to_string**, **float_to_string**, **paquete_to_string**, **pet_to_string**, **res_to_string**, **string_to_pet** y **string_to_res**). Con esto evitamos que la librería dependa de un lenguaje en específico y la comunicación entre máquinas sea realmente independiente. La implementación de todas estas funciones se encuentra en **sock.c**.

En **servidor-sock.c** está implementado el servidor con la gestión interna de los hilos para que sea un servicio concurrente. El hilo principal (**main**) comprueba que los parámetros de ejecución son correctos (**./servidor <num_puerto>**), inicializa el socket para la comunicación (**mySocket**), acepta la comunicación con un cliente (con **accept**) y crea un thread pasando un puntero (para evitar condición de carrera) al descriptor de socket con el cliente (**sd_client_ptr**).

Cada thread realiza la función **worker**: recibe un string del cliente (en **buff**), lo convierte en **Peticion**, bloquea el mutex **mutex_db**, realiza la operación correspondiente según **cod_op**, desbloquea **mutex_db**, envía la **Respuesta** en forma de string y cierra la conexión.

En **proxy-sock.c** están implementadas las funciones de **claves.h** para simular que se ejecutan directamente en el cliente pero realmente se ejecutan en el servidor (al igual que en el ejercicio 1). Hemos implementado dos funciones auxiliares para el proxy que son:

Función	Objetivo
---------	----------

```
int getGlobals();
```

Obtiene los valores para IP_TUPLAS y PORT_TUPLAS

```
int connect_to(int *sd);
```

Conecta con el socket con dirección IP_TUPLAS y puerto PORT_TUPLAS

Las funciones de **claves.h** implementadas en el proxy siguen una estructura similar: preparación previa (obtención de variables de entorno, conexión con servidor, control de valores de parámetros), preparación de **Peticion** (inicializar estructura con los parámetros y pasar estructura a string), envío de **Peticion** con **sock_send**, recepción de un string con **sock_receive** y pasar el string a **Respuesta**.

Fase 2: Compilación

De cara a la compilación, se dispone de un **Makefile**.

En primer lugar, realice **make clean**. Posteriormente, podrá ejecutar **make**.

Antes de ejecutar ningún programa realice los siguientes comandos:

```
export IP_TUPLAS=localhost
export PORT_TUPLAS=8080
```

Para ejecutar el **servidor**, haga:

```
./servidor 8080
```

Para ejecutar el **cliente**, puede hacer:

```
./cliente
```

Puede ejecutar tantos clientes como quiera acompañando la ejecución anterior de tantos `'./cliente'` como clientes quiera ejecutar.

Fase 3: Tests

Los tests realizados en este ejercicio son los mismos que realizamos en el ejercicio 1. Estos son:

T.ID.	Test	Tipo	Función
TC01	int test_set();	Válido	Comprueba creación de nodo sin errores.
TC02	int test_get();	Válido	Comprueba obtención de nodo sin errores.
TC03	int test_modify();	Válido	Comprueba modificación de nodo sin errores.
TC04	int test_delete();	Válido	Comprueba borrar un nodo sin errores.

TC05	<code>int test_exists();</code>	Válido	Comprueba que exista un nodo sin errores.
TC06	<code>int test_duplicate();</code>	Inválido	Comprueba que se devuelve -1 si al crear existe otro nodo igual.
TC07	<code>int test_N_inval();</code>	Inválido	Comprueba que se devuelve -1 si el tamaño no es correcto (>32).
TC08	<code>int test_nonexist();</code>	Inválido	Comprueba que se devuelve -1 si al intentar obtener un nodo no existe.

Además, realizaremos el **TC09** que consiste en realizar un test de carga para comprobar la concurrencia máxima que puede llevar a cabo el servidor en nuestra máquina local. Para ejecutarlo debe ejecutar el **servidor** como se especifica en la **Fase 2** y en otra terminal ejecutar el siguiente script:

```
python3 batch_script.py
```

Nota: Para los tests, es necesario recordar la ejecución del servidor. En particular, el script de batch también fallará en caso de que se nos olvide establecer el servidor y/o las variables de entorno por consola. Se decidió mantener así para no entrar en conflictos en caso de que escogieramos un puerto 'hardcodeado' que otra máquina estuviera usando para otro proceso.

Si se pretende ejecutar directamente, pruebe con:

```
export IP_TUPLAS=localhost
export PORT_TUPLAS=8080
./servidor 8080 &
python3 batch_script.py
```

El resultado, en nuestro caso, alcanza un número alto de clientes antes de encontrar nuestro límite. Sin embargo, este límite se verá limitado por la configuración del SO y hardware. Véase la siguiente imagen:

```
Límite del S0
Fallo
Probando con K = 18184
Límite del S0
Fallo
Probando con K = 18180
Éxito con K = 18180
Probando con K = 18182
Límite del S0
Fallo
Probando con K = 18181
Éxito con K = 18181
Resultado final: Máximo en 18181 ejecuciones concurrentes de cliente
```